

actual boot sequence.

This is the **exact flow** from `start_kernel()` → your GPIO/I2C/SPI driver **probe()**.

Stage 1: Bootloader Loads the Kernel

U-Boot loads:

- `zImage / uImage`
- `Device Tree Blob (.dtb)`
- `initrd (maybe)`

Then jumps to the kernel entry point.

Stage 2: Kernel Entry → `start_kernel()`

The kernel starts at:

`start_kernel()`

This is the heart of booting.

Inside `start_kernel()`, many things happen (in this exact order):

✓ 2.1 Setup architecture

- MMU
- Cache
- Exception vectors
- low-level CPU setup

✓ 2.2 Initialize memory management

- Page allocator
- Slab/slub allocator

✓ 2.3 Initialize interrupt handling

- IRQ subsystem
- Softirqs, tasklets

✓ 2.4 Timers and scheduler

- Timekeeping
- Scheduler activation

✓ 2.5 Enable basic drivers

(e.g., early console, debug UART)

Stage 3: `rest_init()` is called

`start_kernel()` ends by calling:

`rest_init()`

This function creates:

- init process (PID 1)
- kernel threads

One important thing inside is:

`kernel_init()`

This is responsible for:

- Running *all initcalls*
- Mounting rootfs
- Starting user-space

Stage 4: `kernel_init()` → `do_basic_setup()`

`kernel_init()` calls:

`do_basic_setup()`

This is where the REAL GAME starts...

`do_basic_setup()` does:

- ✓ Initializes all subsystems
- ✓ Initializes buses (I2C, SPI, USB, PCI)
- ✓ Runs all **initcalls** (this is key)

Stage 5: Running Initcall Levels (THE MOST IMPORTANT PART)

Linux now executes all initcalls in order, from early → late.

Here is the real order:

1. `early_initcall()`
2. `core_initcall()`
3. `postcore_initcall()`

4. arch_initcall()
5. subsys_initcall()
6. fs_initcall()
7. device_initcall()
8. late_initcall()

This is where subsystems and drivers are registered.

Stage 6: Subsystems Init (subsys_initcall)

Example:

- i2c_init() (I2C framework)
- spi_init() (SPI framework)
- usb_init() (USB core)
- gpio_subsys_init() (GPIO framework)
- regulator_init()

This is VERY IMPORTANT:

Subsystems register themselves BEFORE your driver loads

So:

I2C core loaded → I2C driver registers

SPI core loaded → SPI driver registers

GPIO framework loaded → GPIO driver registers

Stage 7: Device Drivers Register (device_initcall)

Now, Linux runs all driver init functions.

Your driver is normally written as:

```
module_init(my_driver_init);
```

And module_init() expands to:

```
device_initcall(my_driver_init)
```

So now your driver's init function runs:

Example (GPIO):

```
omap_gpio_init_driver() {  
    platform_driver_register(&omap_gpio_driver);  
}
```

This does NOT probe yet.

It only registers the driver structure to the platform bus.

Stage 8: Device Tree (DT) Matching Happens

Now Linux has:

- ✓ All drivers registered
- ✓ All devices (from DT) created

Platform bus goes through:

of/platform/bus.c

And matches devices vs drivers:

- Device Node → compatible
- Driver → of_device_id table

If match → call driver's probe().

Stage 9: Finally! Your Driver's probe() is Called

Once DT node matches with your driver:

```
omap_gpio_probe(struct platform_device *pdev)
```

is called automatically.

Inside probe():

- Get resources
- Map registers with ioremap()
- Request IRQ
- Create sysfs entries
- Register with framework
- Export to /sys/class/gpio

This is the moment your driver "comes alive".

Stage 10: User-Space Starts (init process)

After all initcalls and driver probing, kernel runs user-space:

`execve("/sbin/init")`

→ `systemd`

→ `busybox init`

→ `OpenRC` (depends on distro)

User-space apps can now:

- `open /dev/mydriver`
- access GPIO/I2C/SPI
- talk to your driver

FULL FLOW SUMMARY (Easy to Remember)

`start_kernel()`

↓

`rest_init()`

↓

`kernel_init()`

↓

`do_basic_setup()`

↓

Run initcalls

↓

`subsystems init (I2C, SPI, USB, GPIO)`

↓

`drivers register (platform_driver_register)`

↓

`Device Tree matching`

↓

probe() called



User-space starts

REAL-TIME STORY: Boot Flow → Platform Bus → OMAP GPIO Probe

Example Driver:

drivers/gpio/gpio-omap.c → omap_gpio_driver

Example Device Tree Node:

```
gpio1: gpio@4804c000 {  
    compatible = "ti,omap3-gpio";  
    reg = <0x4804c000 0x1000>;  
    interrupts = <29>;  
    gpio-controller;  
};
```

Power ON → Your Driver Probe()

- 1 Boot ROM runs → loads SPL → loads U-Boot

This is bootloader stage. Nothing to do with your driver yet.

- 2 U-Boot loads Linux kernel + DTB

When U-Boot passes DTB + kernel, your device tree node is already present.

KERNEL STARTS

- 3 start_kernel()

This is the father of all kernel functions.

Inside it, these things happen in order:

✓ setup_arch()

- Parses DTB
- Creates OF nodes for gpio@4804c000

- Fills struct device_node
- Registers platform devices later

✓ mm_init()

- Memory management setup

✓ sched_init()

- Scheduler

✓ init_IRQ()

- Interrupt subsystem online

✓ softirq_init()

✓ timekeeping_init()

✓ rest_init()

This is the important one.

rest_init() internally does:

1. kernel_init() → PID 1
2. kthreadd gets created

INITCALL LEVELS START EXECUTING

This is where YOUR DRIVER REGISTERS.

Run all initcalls

Initcalls are run in this order:

pure_initcall

core_initcall

postcore_initcall

arch_initcall

subsys_initcall <-- OMAP GPIO INIT

fs_initcall

device_initcall

late_initcall

REAL EXAMPLE: OMAP GPIO driver

In actual source:

```
subsys_initcall(omap_gpio_init);
```

So OMAP GPIO runs in subsys_initcall stage.

WHAT HAPPENS in omap_gpio_init()?

Step 1 → Register driver with platform bus

```
platform_driver_register(&omap_gpio_driver);
```

Step 2 → Platform core keeps driver in its list

Nothing probes yet.

5 Device Model (platform bus) matches device

When DT is parsed earlier, the kernel created a platform device:

platform device: "4804c000.gpio"

compatible = "ti,omap3-gpio"

Now platform bus tries to match:

- driver .of_match_table
- device's compatible string

Match found → "ti,omap3-gpio"

6 PROBE() GETS CALLED

This is the moment you have been waiting for.

Kernel calls:

```
omap_gpio_probe(struct platform_device *pdev)
```

Now the story becomes driver-specific.

INSIDE PROBE() – Step by Step (REAL FLOW)

Step 1 → `pdev->dev.of_node`

Read the device tree info:


```
struct device_node *np = pdev->dev.of_node;
```

Step 2 → ioremap() registers

```
res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
```

```
base = devm_ioremap_resource(dev, res);
```

Step 3 → Allocate driver data

```
g = devm_kzalloc(dev, sizeof(*g), GFP_KERNEL);
```

```
platform_set_drvdata(pdev, g);
```

Step 4 → IRQ setup

```
irq = platform_get_irq(pdev, 0);
```

```
devm_request_irq(dev, irq, omap_gpio_irq_handler, ...);
```

Step 5 → Create gpiochip

```
gpiochip_add()
```

Your GPIO controller becomes visible to userspace as:

```
/sys/class/gpio
```

Step 6 → driver probe completes

Driver is ready!

Very Short Summary (Real, Practical)

Stage	Meaning	Example
start_kernel()	Kernel core setup	No driver involved
setup_arch()	Parse DT	Create gpio@4804c000 node
initcalls	Register drivers	omap_gpio_init() called
platform_driver_register	Add driver to list	Add omap_gpio_driver
Device creation	From DT	platform device created
Match	Compare compatible strings "ti,omap3-gpio"	
probe()	Driver startup	Configure registers, IRQ, gpiochip

REAL-TIME STORY IN ONE LINE:

Kernel boots → DT parsed → platform device created → driver registered via initcall → bus matches → your probe() runs.

TARGET DRIVER: omap_gpio

File location (for TI/OMAP/AM335x/BBB):

`drivers/gpio/gpio-omap.c`

FLOW OVERVIEW (Full Story)

`omap_gpio_init_driver()`

↓

`platform_driver_register(&omap_gpio_driver)`

↓

DT matching

↓

`omap_gpio_probe()`

↓

Read DT resources

↓

ioremap GPIO registers

↓

IRQ setup

↓

Export to GPIO subsystem

↓

Driver ready

Now let's go STEP BY STEP with actual kernel code.

STEP 1: Initcall — Driver Enters Boot Process

Inside gpio-omap.c:

```
static int __init omap_gpio_init_driver(void){  
    return platform_driver_register(&omap_gpio_driver);  
}  
subsys_initcall(omap_gpio_init_driver);
```

What happens here?

- ✓ omap_gpio_init_driver() is registered as a subsys_initcall.
- ✓ During boot, Linux runs all initcalls.
- ✓ When it reaches subsys_initcall, it calls this function.
- ✓ This registers the OMAP GPIO driver with the platform bus.

Still no probe() yet! This only registers the driver.

STEP 2: Driver Structure (platform_driver)

```
static struct platform_driver omap_gpio_driver = {  
    .probe = omap_gpio_probe,  
    .remove = omap_gpio_remove,  
    .driver = {  
        .name = "omap-gpio",  
        .of_match_table = omap_gpio_of_match,  
    },  
};
```

Important points:

- .probe = omap_gpio_probe → This is your real driver logic.
 - of_match_table → Used for DT matching.
 - name = "omap-gpio" → Matches platform_device name if needed.
-

STEP 3: Device Tree (DT) Match Table

```
static const struct of_device_id omap_gpio_of_match[] = {  
    { .compatible = "ti,omap3-gpio" },  
    { .compatible = "ti,omap4-gpio" },  
};
```

```
{ .compatible = "ti,am3352-gpio" },
```

```
{ },
```

```
};
```

```
MODULE_DEVICE_TABLE(of, omap_gpio_of_match);
```

These must match nodes from your BBB DT:

Example from am33xx.dtsi:

```
gpio0: gpio@44e07000 {
```

```
    compatible = "ti,am3352-gpio";
```

```
    reg = <0x44e07000 0x1000>;
```

```
    interrupts = <96>;
```

```
    gpio-controller;
```

```
    #gpio-cells = <2>;
```

```
};
```

MATCH FOUND → kernel invokes **probe()**

STEP 4: PROBE FUNCTION (Actual Driver Setup)

Now the REAL driver execution begins:

```
static int omap_gpio_probe(struct platform_device *pdev)
```

```
{
```

```
    struct omap_gpio_chip *chip;
```

```
    struct resource *res;
```

```
    chip = devm_kzalloc(&pdev->dev, sizeof(*chip), GFP_KERNEL);
```

```
    /* Step 1: Get register address */
```

```
    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
```

```
    chip->base = devm_ioremap_resource(&pdev->dev, res);
```

```
    /* Step 2: Get IRQ */
```

```
chip->irq = platform_get_irq(pdev, 0);
```

```
/* Step 3: Init GPIO chip structure */
```

```
chip->chip.direction_input = omap_gpio_input;
```

```
chip->chip.direction_output = omap_gpio_output;
```

```
chip->chip.get = omap_gpio_get;
```

```
chip->chip.set = omap_gpio_set;
```

```
/* Step 4: Register with gpio framework */
```

```
gpiochip_add_data(&chip->chip, chip);
```

```
/* Step 5: Request IRQ handler */
```

```
request_irq(chip->irq, omap_gpio_irq_handler, IRQF_SHARED,
```

```
dev_name(&pdev->dev), chip);
```

```
return 0;
```

```
}
```

Let's break this down like a firmware story.

WHAT EXACTLY HAPPENS INSIDE PROBE()

✓ STEP 4.1: Allocate driver private struct

The driver allocates memory for managing 32 GPIO pins of this bank.

```
chip = devm_kzalloc();
```

✓ STEP 4.2: Map hardware registers (I/O region)

Driver reads the DT:

```
reg = <0x44e07000 0x1000>;
```

and maps it into kernel virtual memory:

```
chip->base = ioremap(0x44e07000, 0x1000);
```

Now driver can access:

- GPIO_OE
- GPIO_DATAIN
- GPIO_DATAOUT
- GPIO_SETDATAOUT
- GPIO_CLEARDATAOUT
- IRQSTATUS registers

✓ STEP 4.3: Get the IRQ number

Example:

```
interrupts = <96>;
```

Driver gets IRQ = 96.

```
chip->irq = platform_get_irq();
```

✓ STEP 4.4: Register GPIO chip with kernel

```
gpiochip_add_data(&chip->chip, chip);
```

This makes your GPIO visible to the kernel:

- /sys/class/gpio/
- gpiolib access from other drivers
- pinctrl and irqchip integration

✓ STEP 4.5: Set IRQ Handler

```
request_irq(96, handler, ...);
```

Now the driver can catch pin interrupts.

Results of Probe()

After probe:

- GPIO registers mapped
- Interrupt handler installed
- GPIO controller registered

- Kernel knows this bank has 32 pins
- Any `gpio_request()` works
- Any `echo 48 > /sys/class/gpio/export` works

Your driver is ACTIVE.

SAME FLOW FOR omap_i2c (Short Version)

To compare, here is quick flow for I2C:

initcall:

```
subsys_initcall(omap_i2c_init_driver);
```

register driver:

```
platform_driver_register(&omap_i2c_driver);
```

DT match:

```
{ .compatible = "ti,omap4-i2c", ... }
```

probe():

- `ioremap` I2C registers
- request IRQs
- configure clock dividers
- register adapter with I2C core
- appear as `/dev/i2c-1`